CSE235 Week 2: Variables, Primitive Types, Arithmetic

This week we'll discuss the basics of variables, primitive data types, and arithmetic operations we can perform on them. We'll also discuss reading input from the console.

Variables

As discussed last week, variables in Python are not declared. Whenever we need to store a value, we pick a name and assign it that value. Variable names in Python use letters, numbers, and underscores. The accepted style for names in Python is `snake_case`, where the words in the name are all lowercase and separated by underscores. Python does not have constants like other languages. All variables are mutable, no matter what. However, if a variable is intended to be constant, programmers typically write it in `SCREAMING_SNAKE_CASE`, where the words in the name are uppercase and separated by underscores. This tells people reading and modifying the code that you intend for that value to be used as a constant. These styles are not rules that the interpreter will enforce, but makes your code consistent with how most people write Python.

Built-in Data Types

Python supports object-oriented programming, meaning it allows for programmer-defined data types. For now, we'll only consider the built-in data types. These are the most basic data types. Below is a table listing the data types:

Type	Description	Example
int	A whole number, can be positive, negative, or 0	x = 4
float	A number with a decimal place. All integers can be stored as a float, but not all floats can be ints	x = -3.2
bool	A value that is `True` or `False`. These are capitalized in Python	x = True
str	A text string. Strings can be in single or double quotes. A set of three double quotes can be used to create a multi-line string	x = 'hello world'
complex	Used to store complex (imaginary) numbers	x = 1.23 + 4.5j

The `list`, `tuple`, and `dict` types are also built in, but we'll discuss them in later weeks. We also will not use `complex` in this class, but it is included here for completeness.

Arithmetic and Operations

We'll discuss the operations for each category of type: numeric, boolean, and string. We'll also discuss the instances where we need to perform a type cast to perform an operation. A type cast tells the interpreter to try to convert a value to another data type.

Numeric

Below is a table describing the arithmetic operations that can be performed on floats and ints.

Operator	Description	Example

Operator	Description	Example
+	Adds two numbers	`x = 1 + 4 # x is 5`
-	Subtracts two numbers	`x = 5.5 - 2 # x is 3.5`
*	Multiplies two numbers	`x = 3 * 8 # x is 24`
/	Divides two numbers (float result)	`x = 1 / 2 # x is 0.5`
//	Floor division of two numbers (int result, always rounds down)	`x = 3 // 4 # x is 0`
%	Remainder of dividing two numbers	`x = 5 % 3 # x is 2`
**	Power operator	`x = 3 ** 2 # x is 9`

There are also bit shift operations that can be performed on numbers, but those are not needed in this class.

Boolean and Comparison

The following operators are used for boolean expressions and comparisons:

Operator	Description	Example
<	Returns true if the left operand is less than the right	`1 < 2 # True`

Operator	Description	Example
>	Returns true if the left operand is greater than the right	`1 > 2 # False`
<=	Returns true if the left operand is less than or equal to the right	`1 <= 2 # True`
>=	Returns true if the left operand is greater than or equal to the right	`1 >= 2 # False`
==	Returns true if the left operand is the same as the right	`1 == 2 # False`
!=	Returns true if the left operand is different from the right	`1 != 2 # True`
and	Returns true if both operands are true	`1 < 2 and 1 == 2 # False since 1 == 2 is False`
or	Returns true if either operand is true	`1 < 2 or 1 == 2 # True since 1 < 2 is True`
not	Negates a Boolean expression	`not 1 == 2 # True, 1 == 2 is False, not False is True`
in	Checks if value is in a collection	`"hello" in "hello world" # True`

Operator	Description	Example
not in	Checks if value is not in a collection	`"hello" not in "hello world" # True`

There are also the `is` and `is not` operators, but we'll discuss them when we talk about classes.

String

Strings have the following operations:

Operator	Description	Example
+	Concatenate two strings	`x = "hello" + "world" # x is "helloworld"`
[i]	Access a specific character	`x = "hello"[0] # x is "h"`
[i:j]	Access a range of characters. The end index is not included	`x = "hello"[0:2] # x is "he"`
Space	Strings that only have space between them are implicitly concatenated	`x = "hello" "world" # x is "helloworld"`

We'll discuss the member functions for the strings in a later week.

Type Casting

Python does not allow you to perform operations between incompatible types. Int and float are compatible with each other, so you can freely combine ints and floats using operators. However, strings and numbers are not compatible. You have to perform a type cast to convert

one of the operands, so both operands are the same type. For example, `x = 60 + "2"` is invalid. You cannot add a string to a number. The interpreter has no way of knowing if you want the string `"602"` as a result, or the number `62`. Therefore, you need to tell it which. To get the result `"602"`, you would cast the value `60` as string: `x = str(60) + "2"`. To get `62`, you'd cast the string `"2"` as an integer: `x = 60 + int("2")`. The name of each type used as a function casts its argument to that type. Below are some examples:

```
PI = float("3.14")
pi_string = "pi = " + str(PI)
radius = int("3")
area = PI * radius ** 2
area_string = "area is " + str(area)
```

Reading User Input

To read input from the terminal, the `input()` function is used. The `input()` function can optionally be passed a message that is printed as a prompt to the terminal. The prompt does not have any additional formatting when it is printed, so make sure to add a space or some other delimiter after the prompt message.

Code:

```
x = input("Enter a number")
y = input("Enter another number: ")
```

Output: (numbers are user input)

```
Enter a number3
Enter another number: 2
```

Notice that there is no space between the word "number" and 3. You have to manually insert whatever formatting you want.

Python only reads user input as a string. If you want it to be another data type, you will need to cast it. In the example above, `x` and `y` hold the strings `"3"` and `"2"`. If we wanted to read them in as ints, then we would need to cast the call to `input()`.

```
x = int(input("Enter a number: "))
y = int(input("Enter another number: "))
```

If the input cannot be cast, say, if "x" were entered instead of an integer number, an exception would be raised and the program would crash. Gracefully handling invalid input data is a topic for a later week.

Example

Below is an example of a program that computes the tax for a purchase.

```
# the tax rate should be constant
# so we make it all uppercase
TAX_RATE = 0.07

# read the item price. we need to cast it as a float
item_price = float(input("Enter the item price: $"))
# tax amount is the price times the tax rate
tax_amount = item_price * TAX_RATE
# total is the price plus the tax
total = tax_amount + item_price

# print the subtotal, tax, and total
# we can print values with commas
print("Subtotal: $", item_price, sep="")
print("Tax:      $", tax_amount, sep="")
print("Total:    $", total, sep="")
```

Example Output:

```
Enter the item price: $399.99
Subtotal: $399.99
Tax:      $27.9993
Total:    $427.9893
```

In a couple of weeks when we discuss functions, we'll discuss ways to clean up the output, so the numbers are rounded properly.

Summary

Variables in Python are written using the `snake_case` style. Python does not have constants, so values that are intended to be constant are written using `SCREAMING_SNAKE_CASE` so other programmers understand your intent. Python offers int, float, bool, string, and complex types. The complex type is not relevant to this class. Python supports basic arithmetic operations for numeric types, string concatenation, and boolean expressions. Some types, like strings and numbers, are not compatible to be combined using operators. Therefore, you will need to type cast one of the operands to tell the interpreter how to process the operation. Finally, we discussed reading data from the user. Python reads all data as strings, so if you want the input to be a different data type, you will have to cast it.
